A6B2_EXPT-9: WHITE BOX TESTING

Basis Path Testing with JUnit - Java

Particular	Details
Course	Software Engineering Lab (24CS01PR0502)
Experiment	9 - White Box Testing
Name	Vineet Shah
Class / Batch	5S CSE / A6 - B2
Roll No.	28
Date of execution	26-10-2026
Case study	Unified Student Placement System (USPS)
Faculty	D A Borikar

Problem Statement [in Java]

[1] Implement a Class as mentioned during the lab instructions. Create a JUnit Test Suite incorporating all possible test cases covering independent paths.

[2] Choose any class from your Class Diagram (one which involves substantial computational logic) and implement it. Now create JUnit suite to test this class appropriately.

Aim

To implement Java classes, draw the control flow graph of each method under test, compute its cyclomatic complexity, identify the independent paths, and create a JUnit test suite with one test case for every independent path.

Theory

White box testing designs test cases from the internal structure of the code. In basis path testing the code is converted into a control flow graph (CFG): every node is a statement or a condition and every edge is a possible transfer of control. A compound condition such as (a <= 0 || b <= 0) is split into one node per simple condition, because Java evaluates it with short-circuiting.

- Cyclomatic complexity V(G) = E - N + 2P, where E = edges, N = nodes and P = connected components (P = 1 for one method).
- Equivalently, V(G) = number of predicate (decision) nodes + 1.
- V(G) is the number of linearly independent paths; each independent path adds at least one edge not covered by the earlier paths. Testing all of them executes every statement and every branch.
- JUnit 5: @Test marks a test method, assertEquals(expected, actual) checks the result, @DisplayName names the test, and @Suite with @SelectClasses groups test classes into a test suite.

Tools Used

Tool	Version / use
Java	Eclipse Temurin JDK 21.0.12 (javac / java)
JUnit	JUnit 5.11.4 (Jupiter) with JUnit Platform 1.11.4 Suite engine, run through the junit-platform-console-standalone launcher
JaCoCo	0.8.12 - records which lines and branches were executed by the test suite
Run script	run_tests.ps1 - compiles src/ and test/, runs WhiteBoxTestSuite, writes reports/

Part 1: Triangle Class

Triangle.classify(a, b, c) takes the three sides of a triangle and returns INVALID (a side is zero or negative), NOT A TRIANGLE (triangle inequality fails), EQUILATERAL, ISOSCELES or SCALENE.

1.1 Source code - Triangle.java

```
1  /**
2   * Part 1: classifies a triangle from the lengths of its three sides.
3   */
4  public class Triangle {
5
6      public static String classify(int a, int b, int c) {
7          if (a <= 0 || b <= 0 || c <= 0) {
8              return "INVALID";
9          }
10         if (a + b <= c || b + c <= a || a + c <= b) {
11             return "NOT A TRIANGLE";
12         }
13         if (a == b) {
14             if (b == c) {
15                 return "EQUILATERAL";
16             }
17             return "ISOSCELES";
18         }
19         if (b == c || a == c) {
20             return "ISOSCELES";
21         }
22         return "SCALENE";
23     }
24 }
```

1.2 Control Flow Graph

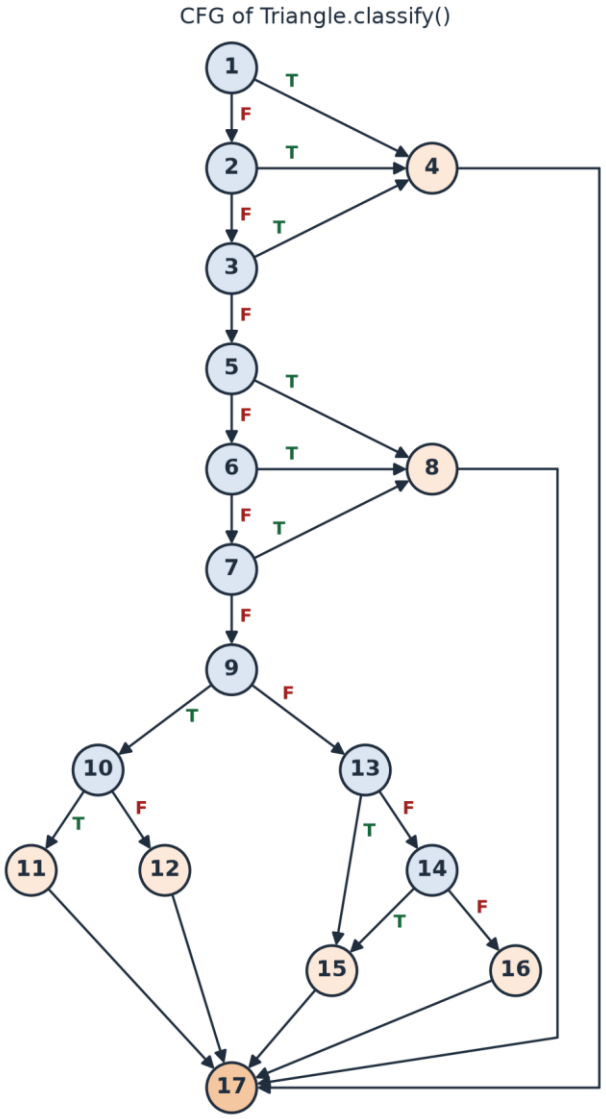


Figure 1: CFG of Triangle.classify() (blue = decision node, orange = return / exit node)

Node	Line	Statement / condition
1	7	a <= 0
2	7	b <= 0
3	7	c <= 0
4	8	return "INVALID"
5	10	a + b <= c
6	10	b + c <= a
7	10	a + c <= b
8	11	return "NOT A TRIANGLE"
9	13	a == b
10	14	b == c
11	15	return "EQUILATERAL"
12	17	return "ISOSCELES"
13	19	b == c
14	19	a == c
15	20	return "ISOSCELES"
16	22	return "SCALENE"
17	23	end of method (exit)

1.3 Cyclomatic Complexity

Method	Calculation	V(G)
E - N + 2P	E = 26 edges, N = 17 nodes, P = 1 -> 26 - 17 + 2	11
Predicate nodes + 1	Decision nodes 1, 2, 3, 5, 6, 7, 9, 10, 13, 14 = 10 -> 10 + 1	11
JaCoCo report	Complexity reported for classify(int, int, int)	11

Hence there are 11 independent paths and at least 11 test cases are required.

1.4 Independent Paths and Test Cases

Path	Node sequence	Input (a, b, c)	Expected output	Actual output	Result
P1	1-4-17	(0, 5, 5)	INVALID	INVALID	**PASS**
P2	1-2-4-17	(4, 0, 3)	INVALID	INVALID	**PASS**
P3	1-2-3-4-17	(4, 3, -2)	INVALID	INVALID	**PASS**
P4	1-2-3-5-8-17	(1, 2, 3)	NOT A TRIANGLE	NOT A TRIANGLE	**PASS**
P5	1-2-3-5-6-8-17	(7, 2, 3)	NOT A TRIANGLE	NOT A TRIANGLE	**PASS**
P6	1-2-3-5-6-7-8-17	(2, 8, 3)	NOT A TRIANGLE	NOT A TRIANGLE	**PASS**
P7	1-2-3-5-6-7-9-10-11-17	(5, 5, 5)	EQUILATERAL	EQUILATERAL	**PASS**
P8	1-2-3-5-6-7-9-10-12-17	(5, 5, 8)	ISOSCELES	ISOSCELES	**PASS**
P9	1-2-3-5-6-7-9-13-15-17	(4, 6, 6)	ISOSCELES	ISOSCELES	**PASS**
P10	1-2-3-5-6-7-9-13-14-15-17	(6, 4, 6)	ISOSCELES	ISOSCELES	**PASS**
P11	1-2-3-5-6-7-9-13-14-16-17	(3, 4, 5)	SCALENE	SCALENE	**PASS**

Actual output is the value returned by classify() during the JUnit run; assertEquals passed for every path (see the console output in Section 3).

1.5 JUnit test class - TriangleTest.java

```
1  import static org.junit.jupiter.api.Assertions.assertEquals;
2
3  import org.junit.jupiter.api.DisplayName;
4  import org.junit.jupiter.api.MethodOrderer;
5  import org.junit.jupiter.api.Order;
6  import org.junit.jupiter.api.Test;
7  import org.junit.jupiter.api.TestMethodOrder;
8
9  /**
10  * Basis-path tests for Triangle.classify(): one test per independent path
11  * of the control flow graph (V(G) = 11). Node numbers refer to the CFG.
12  */
13 @DisplayName("Part 1 - Triangle.classify() independent paths")
14 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
15 class TriangleTest {
16
17     @Test
18     @Order(1)
19     @DisplayName("P1: 1-4-17  side a <= 0")
20     void path1SideANotPositive() {
21         assertEquals("INVALID", Triangle.classify(0, 5, 5));
22     }
23
24     @Test
25     @Order(2)
26     @DisplayName("P2: 1-2-4-17  side b <= 0")
27     void path2SideBNotPositive() {
28         assertEquals("INVALID", Triangle.classify(4, 0, 3));
29     }
30
31     @Test
32     @Order(3)
33     @DisplayName("P3: 1-2-3-4-17  side c <= 0")
34     void path3SideCNotPositive() {
35         assertEquals("INVALID", Triangle.classify(4, 3, -2));
36     }
37
38     @Test
39     @Order(4)
40     @DisplayName("P4: 1-2-3-5-8-17  a + b <= c")
41     void path4FirstInequalityFails() {
```

```
42          assertEquals("NOT A TRIANGLE", Triangle.classify(1, 2, 3));
43      }
44
45      @Test
46      @Order(5)
47      @DisplayName("P5: 1-2-3-5-6-8-17  b + c <= a")
48      void path5SecondInequalityFails() {
49          assertEquals("NOT A TRIANGLE", Triangle.classify(7, 2, 3));
50      }
51
52      @Test
53      @Order(6)
54      @DisplayName("P6: 1-2-3-5-6-7-8-17  a + c <= b")
55      void path6ThirdInequalityFails() {
56          assertEquals("NOT A TRIANGLE", Triangle.classify(2, 8, 3));
57      }
58
59      @Test
60      @Order(7)
61      @DisplayName("P7: 1-2-3-5-6-7-9-10-11-17  a == b == c")
62      void path7Equilateral() {
63          assertEquals("EQUILATERAL", Triangle.classify(5, 5, 5));
64      }
65
66      @Test
67      @Order(8)
68      @DisplayName("P8: 1-2-3-5-6-7-9-10-12-17  a == b, b != c")
69      void path8IsoscelesAEqualsB() {
70          assertEquals("ISOSCELES", Triangle.classify(5, 5, 8));
71      }
72
73      @Test
74      @Order(9)
75      @DisplayName("P9: 1-2-3-5-6-7-9-13-15-17  b == c")
76      void path9IsoscelesBEqualsC() {
77          assertEquals("ISOSCELES", Triangle.classify(4, 6, 6));
78      }
79
80      @Test
81      @Order(10)
82      @DisplayName("P10: 1-2-3-5-6-7-9-13-14-15-17  a == c")
83      void path10IsoscelesAEqualsC() {
84          assertEquals("ISOSCELES", Triangle.classify(6, 4, 6));
85      }
86
87      @Test
88      @Order(11)
89      @DisplayName("P11: 1-2-3-5-6-7-9-13-14-16-17  all sides different")
90      void path11Scalene() {
91          assertEquals("SCALENE", Triangle.classify(3, 4, 5));
92      }
93  }
```

Part 2: EligibilityCriteria Class (from the Class Diagram)

From the class diagram of the Unified Student Placement System (Expt-3), the EligibilityCriteria class is chosen because it holds the main computational logic of the placement process (SRS FR-13, FR-14, BR-03): it turns the allowed-branches text of a job (attribute allowedBranches : String) into a checked list, and for a student it works out every criterion that is not met - minimum CGPA, allowed branch, maximum active backlogs and batch - with a reason for each (operations evaluate() and getReasons() of the diagram). A criterion that is not set is not checked. It uses the Student class, which holds the academic data of a student. The same rules run in the USPS web prototype of the PBL (app/rules.py, in Python); here the class is implemented in Java for JUnit.

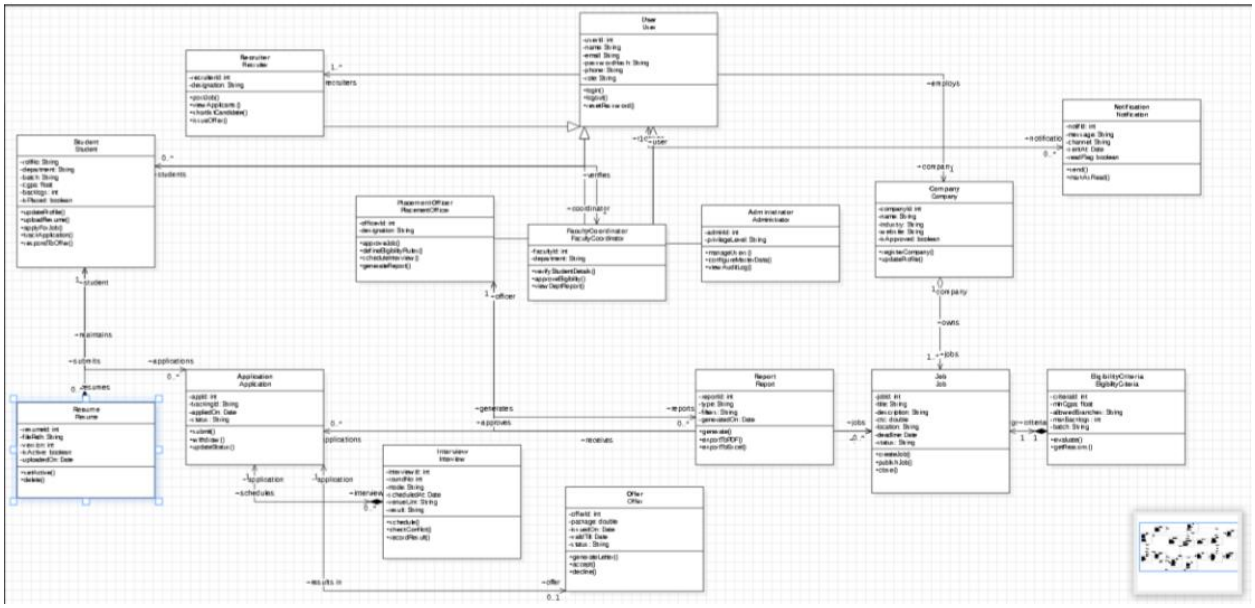


Figure 2: Class diagram from my Expt-3 submission - EligibilityCriteria is linked to Job

2.1 Source code - Student.java

```
 1  /**
 2   * A student of the Unified Student Placement System (class Student of the Expt-3 class
diagram),
 3   * with the academic data used by the eligibility check.
 4   */
 5  public class Student {
 6
 7      private final String rollNo;
 8      private final String department;
 9      private final String batch;
10      private final double cgpa;
11      private final int backlogs;
12
13      public Student(String rollNo, String department, String batch, double cgpa, int backlogs) {
14          this.rollNo = rollNo;
15          this.department = department;
16          this.batch = batch;
17          this.cgpa = cgpa;
18          this.backlogs = backlogs;
19      }
20
21      public String getRollNo() {
22          return rollNo;
23      }
24
25      public String getDepartment() {
26          return department;
27      }
28
29      public String getBatch() {
```

```
        return batch;
    }

    public double getCgpa() {
        return cgpa;
    }

    public int getBacklogs() {
        return backlogs;
    }
}
```

2.2 Source code - EligibilityCriteria.java

```
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * Part 2: eligibility rule-set of a job (class EligibilityCriteria of the Expt-3 class diagram of the
 * Unified Student Placement System, SRS FR-13 / FR-14, BR-03). A criterion that is not set (null or
 * no branches) is not checked. The PBL prototype applies the same rules in Python (app/rules.py).
 */
public class EligibilityCriteria {

    public static final List<String> BRANCHES = List.of("CSE", "IT", "ECE", "EE", "ME", "CE");

    private final int criteriaId;
    private final Double minCgpa;
    private final List<String> allowedBranches;
    private final Integer maxBacklogs;
    private final String batch;

    public EligibilityCriteria(int criteriaId, Double minCgpa, String allowedBranches, Integer maxBacklogs,
                               String batch) {
        this.criteriaId = criteriaId;
        this.minCgpa = minCgpa;
        this.allowedBranches = parseBranches(allowedBranches);
        this.maxBacklogs = maxBacklogs;
        this.batch = (batch == null || batch.isBlank()) ? null : batch.trim();
    }

    /** Turns "cse, IT" into [CSE, IT]: blanks are skipped, duplicates kept once, unknown branches refused.
*/
    public static List<String> parseBranches(String text) {
        List<String> branches = new ArrayList<>();
        if (text == null) {
            return branches;
        }
        for (String part : text.split(",")) {
            String branch = part.trim().toUpperCase(Locale.ROOT);
            if (branch.isEmpty()) {
                continue;
            }
            if (!BRANCHES.contains(branch)) {
                throw new IllegalArgumentException("Unknown branch " + branch + ".");
            }
            if (!branches.contains(branch)) {
                branches.add(branch);
            }
        }
        return branches;
    }

    /** getReasons(): every criterion the student does not meet, in the order CGPA, branch, backlogs, batch.
*/
    public List<String> getReasons(Student s) {
        List<String> reasons = new ArrayList<>();
        if (minCgpa != null) {
            if (s.getCgpa() < minCgpa) {
                reasons.add(String.format(Locale.ROOT, "CGPA %.2f is below the minimum %.2f.", s.getCgpa(),
minCgpa));
            }
        }
        if (!allowedBranches.isEmpty()) {
            if (!allowedBranches.contains(s.getDepartment())) {
                reasons.add("Branch " + s.getDepartment() + " is not in the allowed branches ("
                        + String.join(", ", allowedBranches) + ").");
            }
```

```
63          }
64          if (maxBacklogs != null) {
65              if (s.getBacklogs() > maxBacklogs) {
66                  reasons.add("Active backlogs " + s.getBacklogs() + " exceed the maximum " + maxBacklogs +
"."");
67              }
68          }
69          if (batch != null) {
70              if (!batch.equals(s.getBatch())) {
71                  reasons.add("Batch " + s.getBatch() + " does not match the required batch " + batch + ".");
72              }
73          }
74          return reasons;
75      }
76
77      /** evaluate(): the student is eligible when no criterion is unmet. */
78      public boolean evaluate(Student s) {
79          return getReasons(s).isEmpty();
80      }
81
82      public int getCriteriaId() {
83          return criteriaId;
84      }
85
86      public List<String> getAllowedBranches() {
87          return allowedBranches;
88      }
89  }
```

2.3 Method parseBranches(text)

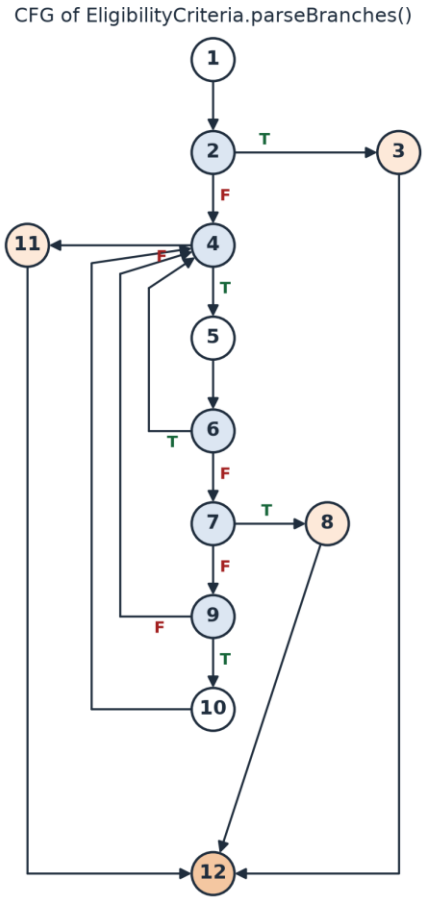


Figure 3: CFG of EligibilityCriteria.parseBranches()

Node	Line	Statement / condition
1	31	branches = new ArrayList<>()
2	32	text == null
3	33	return branches (empty list)
4	35	for (part : text.split(",")) - another part?
5	36	branch = part.trim().toUpperCase()
6	37	branch.isEmpty() (true: continue, line 38)
7	40	!BRANCHES.contains(branch)
8	41	throw IllegalArgumentException("Unknown branch ...")
9	43	!branches.contains(branch)
10	44	branches.add(branch)
11	47	return branches
12	48	end of method (exit)

Method	Calculation	V(G)
E - N + 2P	E = 16, N = 12, P = 1 -> 16 - 12 + 2	6
Predicate nodes + 1	Decision nodes 2, 4, 6, 7, 9 = 5 -> 5 + 1	6
JaCoCo report	Complexity reported for parseBranches(String)	6

Of the 6 basis paths, 1-2-4-11-12 (loop condition false on the first check) is infeasible: String.split() always returns at least one element, so when text is not null the loop body runs at least once. The 5 feasible paths R1-R5 still execute every node and every edge of the graph (edge 4-11 is taken at the end of R2, R4 and R5), and JaCoCo shows 10 of 10 branches covered.

Path	Node sequence	Input text	Expected	Actual	Result
R1	1-2-3-12	null	[] (empty list)	[] (empty list)	**PASS**
R2	1-2-4-5-6-4-11-12	" " (only a blank)	[] (blank skipped)	[] (blank skipped)	**PASS**
R3	1-2-4-5-6-7-8-12	"AI"	IllegalArgumentException: Unknown branch AI.	IllegalArgumentException: Unknown branch AI.	**PASS**
R4	1-2-4-5-6-7-9-10-4-11-12	"CSE"	[CSE]	[CSE]	**PASS**
R5	1-2-4-5-6-7-9-10-4-5-6-7-9-4-11-12	"CSE, cse"	[CSE] (duplicate kept once)	[CSE] (duplicate kept once)	**PASS**
Extra	loop body 5 times	" it , CSE,,ece "	[IT, CSE, ECE]	[IT, CSE, ECE]	**PASS**

2.4 Method getReasons(student)

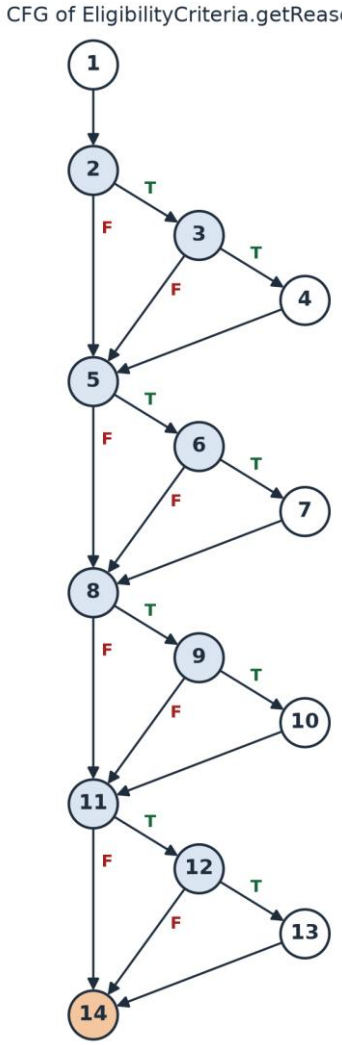


Figure 4: CFG of EligibilityCriteria.getReasons()

Node	Line	Statement / condition
1	52	reasons = new ArrayList<>()
2	53	minCgpa != null
3	54	s.getCgpa() < minCgpa
4	55	add "CGPA x is below the minimum y."
5	58	!allowedBranches.isEmpty()
6	59	!allowedBranches.contains(s.getDepartment())
7	60-61	add "Branch b is not in the allowed branches (...)."
8	64	maxBacklogs != null
9	65	s.getBacklogs() > maxBacklogs
10	66	add "Active backlogs n exceed the maximum m."
11	69	batch != null
12	70	!batch.equals(s.getBatch())
13	71	add "Batch b does not match the required batch r."
14	74	return reasons (exit)

Method	Calculation	V(G)
E - N + 2P	E = 21, N = 14, P = 1 -> 21 - 14 + 2	9
Predicate nodes + 1	Decision nodes 2, 3, 5, 6, 8, 9, 11, 12 = 8 -> 8 + 1	9
JaCoCo report	Complexity reported for getReasons(Student)	9

Test data (demo students of USPS): Aarav - CSE, 2023-2027, CGPA 8.60, 0 backlogs; Kabir - ECE, 2023-2027, CGPA 6.80, 1 backlog; Rohan - ME, 2022-2026, CGPA 7.90, 2 backlogs; Diya - IT, 2023-2027, CGPA 7.00, 0 backlogs. Each test builds its own EligibilityCriteria with only the criteria it needs.

Path	Node sequence	Input (criteria; student)	Expected output	Actual output	Result
A1	1-2-5-8-11-14	no criterion set; Kabir	[] (eligible)	[] (eligible)	**PASS**
A2	1-2-3-5-8-11-14	min CGPA 7.0; Aarav (8.60)	[] (eligible)	[] (eligible)	**PASS**
A3	1-2-3-4-5-8-11-14	min CGPA 7.0; Kabir (6.80)	[CGPA 6.80 is below the minimum 7.00.]	[CGPA 6.80 is below the minimum 7.00.]	**PASS**
A4	1-2-5-6-8-11-14	branches "CSE, IT"; Aarav (CSE)	[] (eligible)	[] (eligible)	**PASS**
A5	1-2-5-6-7-8-11-14	branches "CSE, IT"; Kabir (ECE)	[Branch ECE is not in the allowed branches (CSE, IT).]	[Branch ECE is not in the allowed branches (CSE, IT).]	**PASS**
A6	1-2-5-8-9-11-14	max backlogs 1; Kabir (1)	[] (eligible)	[] (eligible)	**PASS**
A7	1-2-5-8-9-10-11-14	max backlogs 0; Kabir (1)	[Active backlogs 1 exceed the maximum 0.]	[Active backlogs 1 exceed the maximum 0.]	**PASS**
A8	1-2-5-8-11-12-14	batch 2023-2027; Aarav (2023-2027)	[] (eligible)	[] (eligible)	**PASS**
A9	1-2-5-8-11-12-13-14	batch 2023-2027; Rohan (2022-2026)	[Batch 2022-2026 does not match the required batch 2023-2027.]	[Batch 2022-2026 does not match the required batch 2023-2027.]	**PASS**
Extra	1-2-3-4-5-6-7-8-9-10-11-12-13-14	min 8.0, CSE/IT, max 0, 2023-2027; Rohan	4 reasons in the order CGPA, branch, backlogs, batch	4 reasons in the order CGPA, branch, backlogs, batch	**PASS**
Extra	1-2-3-5-8-11-14	min CGPA 7.0; Diya (7.00) - boundary	[] (eligible, CGPA equal to minimum)	[] (eligible, CGPA equal to minimum)	**PASS**

A1-A9 are the 9 independent paths. The extra tests check that all reasons are collected in order when every criterion fails, and the CGPA boundary (equal to the minimum is eligible). evaluate() has V(G) = 1 and is tested with one eligible and one ineligible student; two constructor tests check the parsed branch list and that a blank batch means no batch criterion.

2.5 JUnit test class - EligibilityCriteriaTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

import java.util.List;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.MethodOrderer;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;

/**
 * Basis-path tests for EligibilityCriteria. Node numbers refer to the control flow graphs:
 * parseBranches() has V(G) = 6 and getReasons() has V(G) = 9.
 * Students are the demo students of the USPS prototype.
 */
@DisplayName("Part 2 - EligibilityCriteria independent paths")
class EligibilityCriteriaTest {

    private static final Student AARAV = new Student("R23CS101", "CSE", "2023-2027", 8.60, 0);
    private static final Student KABIR = new Student("R23EC103", "ECE", "2023-2027", 6.80, 1);
    private static final Student ROHAN = new Student("R22ME105", "ME", "2022-2026", 7.90, 2);

    private static EligibilityCriteria rule(Double minCgpa, String branches, Integer maxBacklogs, String
batch) {
        return new EligibilityCriteria(1, minCgpa, branches, maxBacklogs, batch);
    }

    @Nested
    @DisplayName("parseBranches()  V(G) = 6")
    @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
    class ParseBranches {

        @Test
```

```
37          @Order(1)
38          @DisplayName("R1: 1-2-3-12  no branches given (null)")
39          void pathR1Null() {
40              assertEquals(List.of(), EligibilityCriteria.parseBranches(null));
41          }
42
43          @Test
44          @Order(2)
45          @DisplayName("R2: 1-2-4-5-6-4-11-12  only a blank entry is skipped")
46          void pathR2BlankSkipped() {
47              assertEquals(List.of(), EligibilityCriteria.parseBranches("  "));
48          }
49
50          @Test
51          @Order(3)
52          @DisplayName("R3: 1-2-4-5-6-7-8-12  unknown branch refused")
53          void pathR3UnknownBranch() {
54              IllegalArgumentException e = assertThrows(IllegalArgumentException.class,
55                      () -> EligibilityCriteria.parseBranches("AI"));
56              assertEquals("Unknown branch AI.", e.getMessage());
57          }
58
59          @Test
60          @Order(4)
61          @DisplayName("R4: 1-2-4-5-6-7-9-10-4-11-12  one valid branch added")
62          void pathR4OneBranch() {
63              assertEquals(List.of("CSE"), EligibilityCriteria.parseBranches("CSE"));
64          }
65
66          @Test
67          @Order(5)
68          @DisplayName("R5: 1-2-4-...-9-10-4-5-6-7-9-4-11-12  duplicate kept once")
69          void pathR5Duplicate() {
70              assertEquals(List.of("CSE"), EligibilityCriteria.parseBranches("CSE, cse"));
71          }
72
73          @Test
74          @Order(6)
75          @DisplayName("Loop many times: ' it , CSE,,ece ' gives IT, CSE, ECE")
76          void loopManyTimes() {
77              assertEquals(List.of("IT", "CSE", "ECE"), EligibilityCriteria.parseBranches(" it , CSE,,ece "));
78          }
79      }
80
81      @Nested
82      @DisplayName("getReasons()  V(G) = 9")
83      @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
84      class GetReasons {
85
86          @Test
87          @Order(1)
88          @DisplayName("A1: 1-2-5-8-11-14  no criterion set")
89          void pathA1NoCriteria() {
90              assertEquals(List.of(), rule(null, null, null, null).getReasons(KABIR));
91          }
92
93          @Test
94          @Order(2)
95          @DisplayName("A2: 1-2-3-5-8-11-14  CGPA 8.60 meets minimum 7.0")
96          void pathA2CgpaMet() {
97              assertEquals(List.of(), rule(7.0, null, null, null).getReasons(AARAV));
98          }
99
100         @Test
101         @Order(3)
102         @DisplayName("A3: 1-2-3-4-5-8-11-14  CGPA 6.80 below minimum 7.0")
103         void pathA3CgpaBelow() {
104             assertEquals(List.of("CGPA 6.80 is below the minimum 7.00."), rule(7.0, null, null,
null).getReasons(KABIR));
105         }
106
107         @Test
108         @Order(4)
109         @DisplayName("A4: 1-2-5-6-8-11-14  branch CSE allowed")
110         void pathA4BranchAllowed() {
111             assertEquals(List.of(), rule(null, "CSE, IT", null, null).getReasons(AARAV));
112         }
113
114         @Test
115         @Order(5)
116         @DisplayName("A5: 1-2-5-6-7-8-11-14  branch ECE not allowed")
117         void pathA5BranchNotAllowed() {
```

```
118             assertEquals(List.of("Branch ECE is not in the allowed branches (CSE, IT)."),
119                     rule(null, "CSE, IT", null, null).getReasons(KABIR));
120         }
121
122         @Test
123         @Order(6)
124         @DisplayName("A6: 1-2-5-8-9-11-14  1 backlog equals maximum 1")
125         void pathA6BacklogsEqual() {
126             assertEquals(List.of(), rule(null, null, 1, null).getReasons(KABIR));
127         }
128
129         @Test
130         @Order(7)
131         @DisplayName("A7: 1-2-5-8-9-10-11-14  1 backlog exceeds maximum 0")
132         void pathA7BacklogsExceed() {
133             assertEquals(List.of("Active backlogs 1 exceed the maximum 0."), rule(null, null, 0,
null).getReasons(KABIR));
134         }
135
136         @Test
137         @Order(8)
138         @DisplayName("A8: 1-2-5-8-11-12-14  batch 2023-2027 matches")
139         void pathA8BatchMatches() {
140             assertEquals(List.of(), rule(null, null, null, "2023-2027").getReasons(AARAV));
141         }
142
143         @Test
144         @Order(9)
145         @DisplayName("A9: 1-2-5-8-11-12-13-14  batch 2022-2026 does not match")
146         void pathA9BatchDiffers() {
147             assertEquals(List.of("Batch 2022-2026 does not match the required batch 2023-2027."),
148                     rule(null, null, null, "2023-2027").getReasons(ROHAN));
149         }
150
151         @Test
152         @Order(10)
153         @DisplayName("All four criteria fail together (job J-1 rules, student Rohan): reasons in order")
154         void allCriteriaFail() {
155             assertEquals(List.of("CGPA 7.90 is below the minimum 8.00.",
156                             "Branch ME is not in the allowed branches (CSE, IT).",
157                             "Active backlogs 2 exceed the maximum 0.",
158                             "Batch 2022-2026 does not match the required batch 2023-2027."),
159                     rule(8.0, "CSE, IT", 0, "2023-2027").getReasons(ROHAN));
160         }
161
162         @Test
163         @Order(11)
164         @DisplayName("Boundary: CGPA exactly equal to the minimum (7.00 vs 7.0) is eligible")
165         void cgpaEqualsMinimum() {
166             Student diya = new Student("R23IT102", "IT", "2023-2027", 7.00, 0);
167             assertEquals(List.of(), rule(7.0, null, null, null).getReasons(diya));
168         }
169     }
170
171     @Nested
172     @DisplayName("Constructor")
173     class Constructor {
174
175         @Test
176         @DisplayName("Branches text is parsed and the id kept: id 7, ' it, cse ' gives IT, CSE")
177         void branchesParsed() {
178             EligibilityCriteria c = new EligibilityCriteria(7, null, " it, cse ", null, null);
179             assertEquals(7, c.getCriteriaId());
180             assertEquals(List.of("IT", "CSE"), c.getAllowedBranches());
181         }
182
183         @Test
184         @DisplayName("Blank batch '   ' means no batch criterion: Rohan (2022-2026) has no reason")
185         void blankBatchIgnored() {
186             assertEquals(List.of(), rule(null, null, null, "   ").getReasons(ROHAN));
187         }
188     }
189
190     @Nested
191     @DisplayName("evaluate()  V(G) = 1")
192     class Evaluate {
193
194         @Test
195         @DisplayName("Eligible student: Aarav for J-1 (CGPA >= 7.0, CSE/IT, 0 backlogs, 2023-2027)")
196         void eligible() {
197             assertTrue(rule(7.0, "CSE, IT", 0, "2023-2027").evaluate(AARAV));
198         }
```

```
199
200         @Test
201         @DisplayName("Ineligible student: Kabir for J-1")
202         void ineligible() {
203             assertFalse(rule(7.0, "CSE, IT", 0, "2023-2027").evaluate(KABIR));
204         }
205     }
206 }
```

3. JUnit Test Suite and Execution

3.1 Test suite - WhiteBoxTestSuite.java

```java
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;
import org.junit.platform.suite.api.SuiteDisplayName;

/**
 * JUnit test suite for Expt-9: runs the independent-path tests of both parts together.
 */
@Suite
@SuiteDisplayName("Expt-9 White Box Testing Suite")
@SelectClasses({ TriangleTest.class, EligibilityCriteriaTest.class })
public class WhiteBoxTestSuite {
}
```

3.2 Command used

```
powershell -ExecutionPolicy Bypass -File run_tests.ps1

# inside run_tests.ps1 (test run):
java -javaagent:lib\jacoco-0.8.12\lib\jacocoagent.jar=destfile=out\jacoco.exec \
     -jar lib\junit-platform-console-standalone-1.11.4.jar execute \
     --class-path "out\classes;out\test-classes" --select-class WhiteBoxTestSuite \
     --details=tree
```

3.3 Console output (captured from the run on 26-10-2026)

```
╷
├─ JUnit Platform Suite ✓
│  └─ Expt-9 White Box Testing Suite ✓
│     └─ JUnit Jupiter ✓
│        ├─ Part 1 - Triangle.classify() independent paths ✓
│        │  ├─ P1: 1-4-17  side a <= 0 ✓
│        │  ├─ P2: 1-2-4-17  side b <= 0 ✓
│        │  ├─ P3: 1-2-3-4-17  side c <= 0 ✓
│        │  ├─ P4: 1-2-3-5-8-17  a + b <= c ✓
│        │  ├─ P5: 1-2-3-5-6-8-17  b + c <= a ✓
│        │  ├─ P6: 1-2-3-5-6-7-8-17  a + c <= b ✓
│        │  ├─ P7: 1-2-3-5-6-7-9-10-11-17  a == b == c ✓
│        │  ├─ P8: 1-2-3-5-6-7-9-10-12-17  a == b, b != c ✓
│        │  ├─ P9: 1-2-3-5-6-7-9-13-15-17  b == c ✓
│        │  ├─ P10: 1-2-3-5-6-7-9-13-14-15-17  a == c ✓
│        │  └─ P11: 1-2-3-5-6-7-9-13-14-16-17  all sides different ✓
│        └─ Part 2 - EligibilityCriteria independent paths ✓
│           ├─ evaluate()  V(G) = 1 ✓
│           │  ├─ Ineligible student: Kabir for J-1 ✓
│           │  └─ Eligible student: Aarav for J-1 (CGPA >= 7.0, CSE/IT, 0 backlogs, 2023-2027) ✓
│           ├─ Constructor ✓
│           │  ├─ Branches text is parsed and the id kept: id 7, ' it, cse ' gives IT, CSE ✓
│           │  └─ Blank batch '   ' means no batch criterion: Rohan (2022-2026) has no reason ✓
│           ├─ getReasons()  V(G) = 9 ✓
│           │  ├─ A1: 1-2-5-8-11-14  no criterion set ✓
│           │  ├─ A2: 1-2-3-5-8-11-14  CGPA 8.60 meets minimum 7.0 ✓
│           │  ├─ A3: 1-2-3-4-5-8-11-14  CGPA 6.80 below minimum 7.0 ✓
│           │  ├─ A4: 1-2-5-6-8-11-14  branch CSE allowed ✓
│           │  ├─ A5: 1-2-5-6-7-8-11-14  branch ECE not allowed ✓
│           │  ├─ A6: 1-2-5-8-9-11-14  1 backlog equals maximum 1 ✓
│           │  ├─ A7: 1-2-5-8-9-10-11-14  1 backlog exceeds maximum 0 ✓
│           │  ├─ A8: 1-2-5-8-11-12-14  batch 2023-2027 matches ✓
│           │  ├─ A9: 1-2-5-8-11-12-13-14  batch 2022-2026 does not match ✓
│           │  ├─ All four criteria fail together (job J-1 rules, student Rohan): reasons in order ✓
│           │  └─ Boundary: CGPA exactly equal to the minimum (7.00 vs 7.0) is eligible ✓
│           └─ parseBranches()  V(G) = 6 ✓
│              ├─ R1: 1-2-3-12  no branches given (null) ✓
│              ├─ R2: 1-2-4-5-6-4-11-12  only a blank entry is skipped ✓
│              ├─ R3: 1-2-4-5-6-7-8-12  unknown branch refused ✓
│              ├─ R4: 1-2-4-5-6-7-9-10-4-11-12  one valid branch added ✓
```

```
│              ├─ R5: 1-2-4-...-9-10-4-5-6-7-9-4-11-12  duplicate kept once ✓
│              └─ Loop many times: ' it , CSE,,ece ' gives IT, CSE, ECE ✓
├─ JUnit Jupiter ✓
└─ JUnit Vintage ✓

Test run finished after 381 ms
[         11 containers found      ]
[          0 containers skipped    ]
[         11 containers started    ]
[          0 containers aborted    ]
[         11 containers successful ]
[          0 containers failed     ]
[         32 tests found           ]
[          0 tests skipped         ]
[         32 tests started         ]
[          0 tests aborted         ]
[         32 tests successful      ]
[          0 tests failed          ]
```

Tests found	Tests successful	Tests failed
32	32	0

3.4 Code Coverage (JaCoCo)

Class	Lines covered	Branches covered	Complexity covered
Triangle	11 / 12	20 / 20	11 / 12
EligibilityCriteria	38 / 38	30 / 30	22 / 22
Student	11 / 12	no branches	5 / 6

Every line and every branch of EligibilityCriteria is executed by the suite (100%), and every branch of Triangle. The only lines not executed are the implicit default constructor of Triangle (classify() is static, so no object is created) and Student.getRollNo(), which the eligibility check does not use.

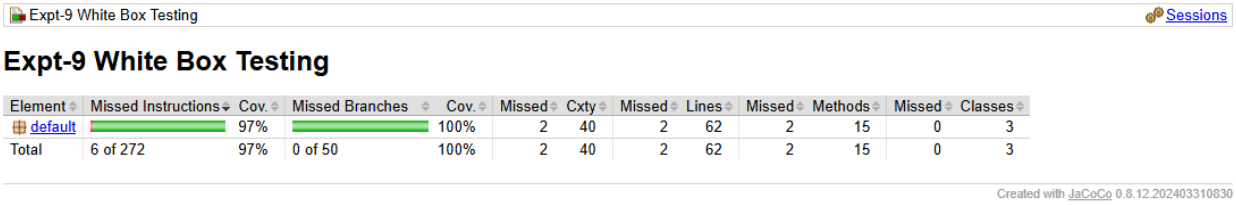


Figure 5: JaCoCo coverage summary for all classes

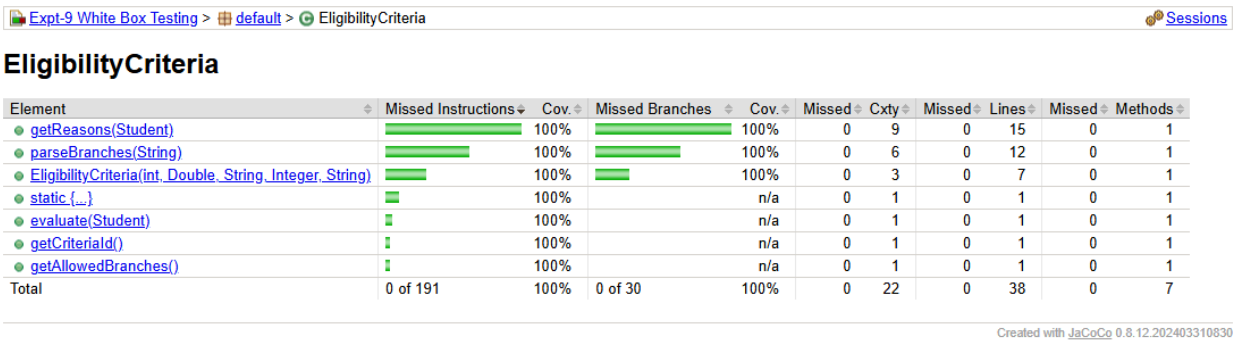


Figure 6: EligibilityCriteria - complexity (Cxty) and coverage per method

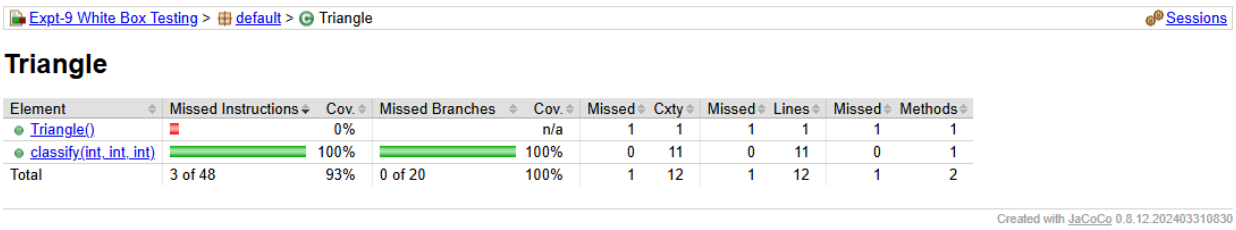


Figure 7: Triangle - complexity (Cxty) and coverage per method

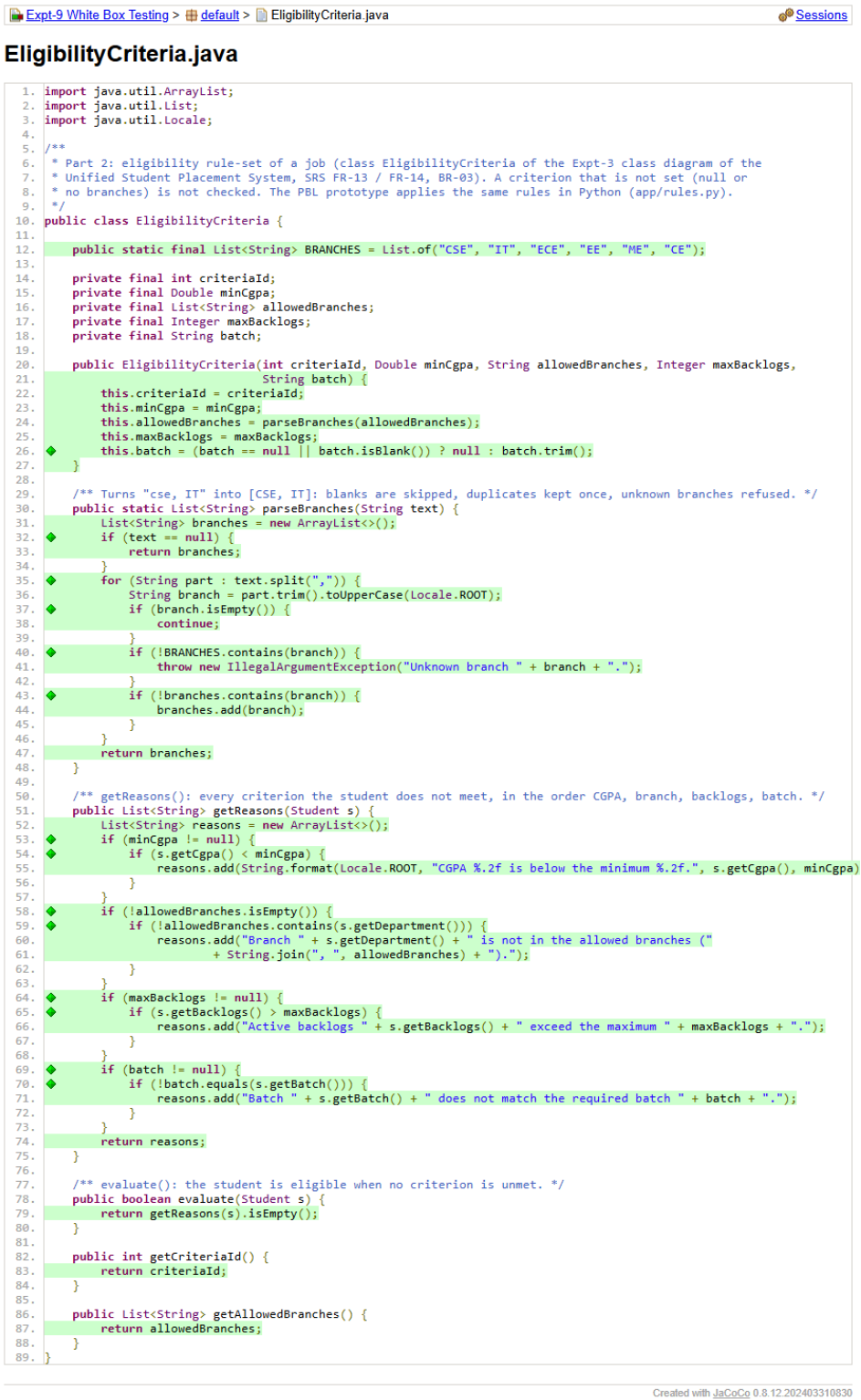


Figure 8: EligibilityCriteria.java in the JaCoCo report - green lines and diamonds show executed code and fully covered branches

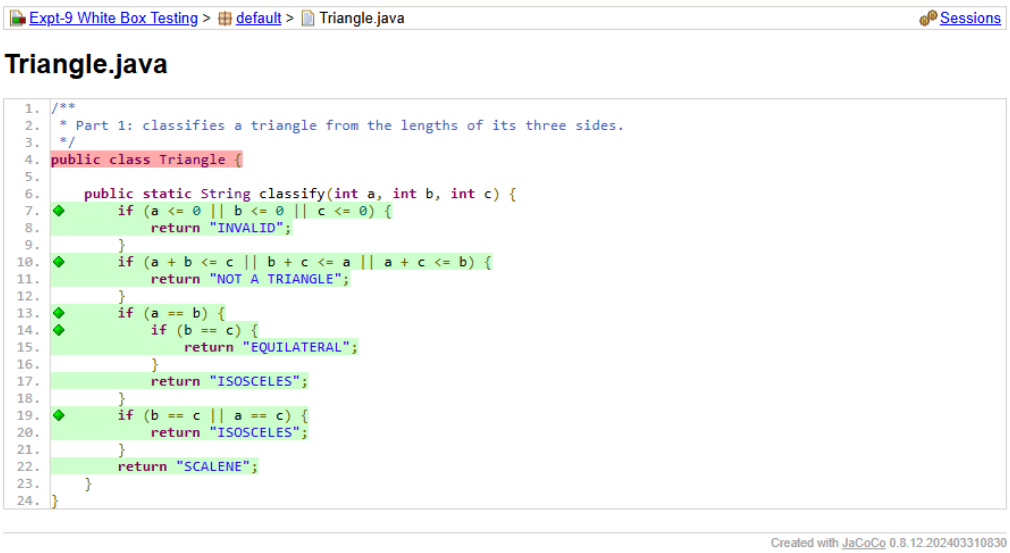


Figure 9: Triangle.java in the JaCoCo report

Conclusion

Basis path testing was applied to Triangle.classify() (V(G) = 11) and to the EligibilityCriteria class of the Unified Student Placement System - parseBranches() (V(G) = 6, one infeasible path) and getReasons() (V(G) = 9). One JUnit test was written for every feasible independent path, plus extra tests for repeated loop execution, the CGPA boundary, evaluate() and the constructor, and all tests were grouped in WhiteBoxTestSuite. The suite ran 32 tests: 32 passed and 0 failed, and JaCoCo confirmed that every branch of both classes was executed.